SAP Fiori Elements

with CAP OData V4

Full Course Review

SESSION 4 • UX405 • 2 HOURS

C_FIORD: Fiori Elements & FPM — 21–30% | Annotations — 11–20% | OData & BTP — 11–20%

UX405: Fiori Elements App Based on CAP OData V4 (14 units, 9h 19min)

learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service

Why UX405 Is the Most Exam-Critical Session

UX405 alone can account for up to ~60% of your C_FIORD exam score

Topic	Weight	Course	Session
SAP Fiori Elements & FPM	21–30%	UX405, UX406, UX410	TODAY ★
Fiori Elements Annotations	11–20%	UX405, UX406	TODAY ★
OData and SAP BTP	11–20%	UX405, UX406	TODAY ★

← *UX405 feeds THREE exam topics. Master this course = strongest return on study time.*

Complete Unit Coverage — Today's Roadmap

All 14 units of UX405

U1	**Fiori Elements Overview** Benefits, architecture, metadata-driven UI	1h 7min	U8	**Multiple Views** Single/multi table mode, @UI.SelectionVariant, side effects	28min
U2	**SAP Fiori Tools** BAS generators, Page Map, Page Editor, Guided Dev	48min	U9	**Object Page Header** KPI, micro charts, header actions, @UI.HeaderInfo	38min
U3	**Fiori Elements Templates** List Report, Object Page, OVP, ALP, Worklist	45min	U10	**Object Page Body** @UI.Facets, FieldGroups, tables, form sections	48min
U4	**General Features** Flexible Column Layout, draft handling, condensed mode	25min	U11	**Dynamic Field Control** Criticality, @Core.Computed, field importance, validation	30min
U5	**List Report Filter Bar** @UI.SelectionFields, filter field config, search	13min	U12	**Flexible Programming Model** Custom sections/columns, Building Blocks (Chart, Table, FilterBar)	1h 0min
U6	**List Report Actions** Generic + custom, bound vs unbound, @UI.DataFieldForAction	45min	U13	**Navigation Concept** Intent-based, internal, external, cross-app navigation	44min
U7	**Table Column Content** Progress indicator, criticality, micro charts, rating	35min	U14	**Analytical List Page** Visual filter, KPI tag, chart + table, @UI.Chart	33min

UX405

Fiori Elements Overview + Tools + Templates

Units 1–3 • What is Fiori Elements, Fiori Tools in BAS, floorplan templates

🎯 EXAM: Fiori Elements & FPM 21–30%

U1: Getting an Overview of SAP Fiori Elements for OData V4

Annotation-driven UI • CAP OData V4 • no hand-coded XML views

What Is SAP Fiori Elements? (U1)

- Metadata-driven UI framework — no hand-coded SAPUI5 views needed
- Apps generated from OData V4 annotations at runtime
- SAP provides pre-built, tested, accessible floorplan templates
- Developer writes annotations; Fiori Elements renders the UI
- Faster development, lower maintenance, consistent UX
- OData V4 ONLY — not compatible with V2 services

CAP — Cloud Application Programming Model (U1)

- SAP framework for building cloud-native apps on BTP
- Supports Node.js and Java runtimes
- CDS (Core Data Services) defines data models & services
- Auto-generates OData V4 service from .cds model files
- Mock server built-in — develop without real backend
- Deployment: SAP BTP (Cloud Foundry, Kyma, or serverless)

Fiori Elements vs Freestyle SAPUI5 — Key Differences

Fiori Elements

- Annotation-driven — no XML views
- Templates by SAP: tested, accessible
- Automatic UX features (draft, side effects)
- Less custom code = lower maintenance

Freestyle SAPUI5

- Hand-coded XML views & controllers
- Full UI flexibility — any layout possible
- All logic must be implemented manually
- Higher maintenance cost, more expertise

When to choose FE

- Transactional / Analytical standard apps
- Must follow SAP UX guidelines
- Team needs fast delivery
- Exam: always prefer FE where annotations fit

🎯 EXAM: Fiori Elements & FPM 21–30%

U2: SAP Fiori Tools + U3: Fiori Elements Templates

BAS tooling • Page Map • Page Editor • all five floorplans

SAP Fiori Tools (U2) — in SAP BAS

Application Generator	Yeoman wizard → scaffolds full Fiori Elements project
Page Map	Visual editor — add/remove pages, set navigation routing
Page Editor	Click-to-configure UI: add columns, filter fields, sections
Guided Development	Step-by-step wizards for advanced features (micro charts etc.)
Service Modeler	Visualise & edit CDS model graphically

Fiori Elements Templates (U3)

List Report + Object Page	Most common — browse list, open detail. Any DB.
Overview Page (OVP)	Cards dashboard — KPIs from multiple data sources
Analytical List Page (ALP)	Chart + table + visual filter. SAP HANA only.
Worklist	Simplified list without complex filters. Any DB.
Form Entry Object Page	Single-entity form for create/edit use cases.

⚠ Exam Trap ★

Fiori Elements ≠ Freestyle SAPUI5. FE generates UI from annotations — you do NOT write XML views. CAP ≠ ABAP Gateway: CAP is Node.js/Java on BTP; Gateway is ABAP-based. All Fiori Elements apps with CAP require OData V4 — NOT V2. Page Editor works on annotations in the background — changes write to annotation files.

UX405

List Report Deep Dive

Units 4–8 • FCL, Filter Bar, Actions, Table Columns, Multiple Views, Side Effects

🎯 EXAM: Annotations 11–20% | FE & FPM 21–30%

U4: General Features + U5: List Report Filter Bar

Flexible Column Layout • draft • @UI.SelectionFields • filter field config

U4: General Features

- Flexible Column Layout (FCL): displays List + Object Page side by side (2 or 3 columns)
- Draft handling: save work-in-progress without committing to backend
- Condensed mode: compact table row height for dense data
- Inline creation: add new row directly in list table without navigating away
- Auto-routing: Page Map configures navigation between pages automatically
- manifest.json: routing config for Fiori Elements managed by Fiori Tools

U5: List Report Filter Bar

- @UI.SelectionFields: defines which fields appear as filters in the filter bar
- Example: @UI.SelectionFields: [SalesOrder, Customer, Status]
- Filter fields link to OData properties — must exist in the entity
- Search enabled via @Search.searchable: true on CDS entity
- Adapt Filter: end users add/remove visible filter fields at runtime
- Default filter values: set via @UI.SelectionVariant annotation

Annotation: @UI.SelectionFields → Filter Bar

```
@UI.SelectionFields: [SalesOrder, SoldToParty, OverallSDProcessStatus]
annotate SalesOrderService.SalesOrder with {
  SalesOrder          @UI.SelectionFields
  SoldToParty         @UI.SelectionFields;
  OverallStatus       @UI.SelectionFields;
};
```

🎯 EXAM: Annotations 11–20% | FE & FPM 21–30%

U6: List Report Actions + U7: Table Column Content

Bound vs unbound actions • @UI.DataFieldForAction • progress indicator • micro charts

U6: Actions — Bound vs Unbound

- Generic actions: built-in FE — Create, Edit, Delete, Save, Discard
- Bound action: acts on a SPECIFIC selected entity instance (needs selection)
- Unbound action: does NOT need a selected entity — independent operation
- @UI.DataFieldForAction: adds a button to toolbar linking to a CAP action
- Critical action: confirmation dialog required — annotate with @Common.IsActionCritical
- Inline action: button rendered directly in each table row (not toolbar)
- Hiding actions: annotate with @UI.Hidden or use @Core.OperationAvailable

U7: Table Column Visualisations

- @UI.DataFieldForAnnotation: embeds a chart/indicator control in a column
- Progress Indicator: @UI.DataPoint with Visualization=#PROGRESS + value/target
- Rating Indicator: @UI.DataPoint with Visualization=#RATING (star display)
- Criticality colouring: annotate property with @UI.DataPoint.Criticality
- 1=Negative(red) 2=Critical(amber) 3=Positive(green) 0=Neutral
- Micro Chart: @UI.Chart embedded inside @UI.DataFieldForAnnotation in @UI.LineItem
- Field Importance: @UI.Importance #HIGH ensures column always visible

⚠ Exam Trap ★

Bound vs Unbound: bound action = requires row selection (acts on that record). Unbound action = runs without selection. Exam will present a scenario — identify which is needed. Criticality values are NUMBERS (1/2/3), not text strings. @UI.DataFieldForAction is used for CUSTOM actions; generic actions (Create/Delete) are automatic.

🎯 EXAM: Annotations 11–20% | FE & FPM 21–30%

U8: Multiple Views in the List Report

Single table / multi table views • @UI.SelectionVariant • side effects

Multiple Views (U8)

- Single table mode: one table with tab bar to switch between filtered views
- Multiple table mode: each tab shows a DIFFERENT entity set / table config
- @UI.SelectionVariant: defines pre-set filter + sort criteria for each tab view
- @UI.SelectionVariant.SelectOptions: filter conditions on specific properties
- @UI.SelectionVariant.Text: label shown on the tab in the UI
- Views configured in manifest.json under 'views' setting of List Report
- Chart + table combo: show chart and table simultaneously on same page

Side Effects (U8)

- Side effect: a field change on the UI triggers automatic backend data refresh
- Example: changing Quantity → Price is recalculated from backend automatically
- Annotated on CDS with @Common.SideEffects annotation
- SourceProperties: which field triggers the side effect
- TargetProperties / TargetEntities: what data to re-fetch from backend
- Saves the user from manually refreshing — happens transparently
- Critical for exam: side effects are ANNOTATIONS, not JavaScript event handlers

@Common.SideEffects — Annotation Example

```
SalesOrder @Common.SideEffects #QuantityChanged: {
  SourceProperties: [RequestedQuantity],
  TargetProperties:  [NetAmount, TaxAmount, GrossAmount]
};
```

UX405

Object Page Deep Dive

Units 9–11 • Header area, body sections, dynamic field control, validation

🎯 EXAM: Annotations 11–20% | FE & FPM 21–30%

U9: Object Page Header + U10: Object Page Body

@UI.HeaderInfo • @UI.Facets • @UI.FieldGroup • connected tables inside object page

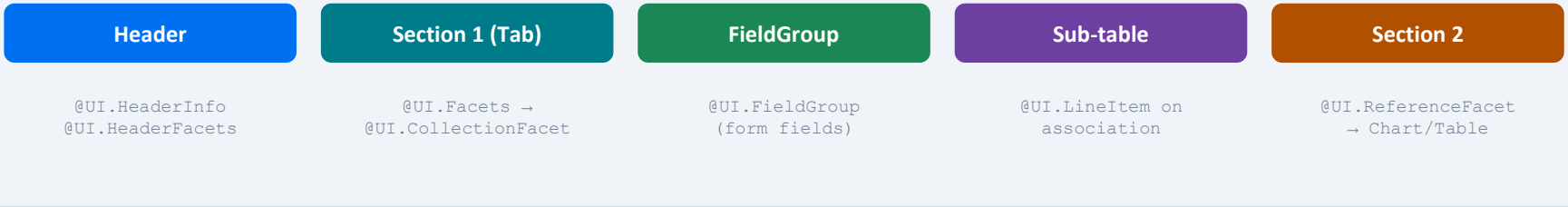
U9: Object Page Header

`@UI.HeaderInfo.TypeName`	Singular entity label: 'Sales Order'
`@UI.HeaderInfo.TypeNamePlural`	Plural label for list: 'Sales Orders'
`@UI.HeaderInfo.Title`	Property shown as main title (e.g. SalesOrderID)
`@UI.HeaderInfo.Description`	Sub-title property (e.g. CustomerName)
`@UI.HeaderFacets`	Embeds KPIs, micro charts, progress in header area
`@UI.DataPoint`	Defines a KPI or metric for display in header

U10: Object Page Body

`@UI.Facets`	Top-level body structure — defines sections/tabs
`@UI.ReferenceFacet`	Points to a FieldGroup, Table, or Chart annotation
`@UI.CollectionFacet`	Groups multiple ReferenceFacets into one section/tab
`@UI.FieldGroup`	Groups fields together inside a form section
`@UI.LineItem (on assoc.)`	Defines columns for an embedded sub-table
`@UI.Hidden`	Hide a field in display but keep in model

Object Page Structure



🎯 EXAM: Annotations 11–20% | FE & FPM 21–30%

U11: Dynamic Field Control + Input Validation

Criticality • field states • @FieldControl • CAP input validation handlers

Dynamic Field Control (U11)

- @FieldControl: controls editability/visibility of a field dynamically
- FieldControl values: 0=Hidden, 1=ReadOnly, 3=Optional, 7=Mandatory
- Value comes from a PROPERTY in the entity (not a static value)
- Example: field is mandatory only when Status = 'Draft'
- @Core.Computed: field is computed backend-side — not editable by user
- @Core.Immutable: field can be set on CREATE but not changed afterwards
- Criticality on field: colours field value (1=red, 2=amber, 3=green)

Input Validation in CAP (U11)

- Validation logic goes in CAP service handlers (Node.js / Java)
- Before-handler: req.before('CREATE', ...) — validate before save
- Use req.error() to raise a validation error — shown in UI message
- req.error(400, 'Quantity must be positive') — 400 = bad request
- Messages shown inline on the field in Fiori Elements automatically
- @mandatory (CDS) = required field — CAP enforces at service level
- Client-side: FE shows immediate feedback before backend call

@FieldControl — Dynamic Editability Example

```
entity SalesOrder {
  Status          : String;                // drives field control
  PurchaseOrder   : String @FieldControl: StatusFieldControl;
  StatusFieldControl : Integer @Core.Computed;  // 7=mandatory, 1=readonly
}
```

UX405

Flexible Programming Model + Navigation + ALP

Units 12–14 • Custom extensions, building blocks, intent-based nav, Analytical List Page

🎯 EXAM: Fiori Elements & FPM 21–30%

U12: Flexible Programming Model (FPM)

Extend FE apps with custom code — without replacing standard templates

What Is FPM? (U12)

- FPM = way to add CUSTOM UI elements to a Fiori Elements app
- Extension points: Custom Section, Custom Column, Custom Action, Custom Header Facet
- Custom code runs alongside generated FE content — not a replacement
- Created as XML Fragments registered in manifest.json
- Controller extension: add custom event handlers without overriding base
- Use FPM when standard annotation config is NOT sufficient
- Key rule: prefer annotations first; use FPM only for what annotations can't do

Building Blocks (U12)

- Building Blocks = reusable FE controls usable in custom sections
- macros:Table — renders FE-managed table in custom section
- macros:Chart — renders FE-managed chart in custom section
- macros:FilterBar — renders FE filter bar in custom section
- macros:Field — renders a single annotated field
- Building Blocks bring FE UX into custom code — consistent look
- Declared as XML namespace: xmlns:macros='sap.fe.macros'

FPM Extension Types at a Glance

Custom Section	Object Page body — additional full section	**Custom Header Facet**	Object Page header — extra widget
Custom Column	List Report or OP table — new table column	**Controller Extension**	Hook into FE lifecycle events
Custom Action	Toolbar button — triggers custom logic		

🎯 EXAM: FE & FPM 21–30% | OData & BTP 11–20%

U13: Navigation Concept + U14: Analytical List Page

Intent-based navigation • external nav • ALP visual filter • @UI.Chart

U13: Navigation in Fiori Elements

- Internal navigation: List Report → Object Page (auto-configured by Fiori Tools)
- Intent-based navigation: semantic object + action (e.g. SalesOrder-displayFactSheet)
- @UI.DataFieldWithIntentBasedNavigation: link that navigates to another FLP app
- External navigation: navigate OUT of current app to a separate FLP tile/app
- @UI.DataFieldWithNavigationPath: navigate to related entity within same service
- Breadcrumb: FE auto-renders back-navigation breadcrumb in Object Page header

U14: Analytical List Page (ALP)

- ALP = chart + table + visual filter on one page — SAP HANA ONLY
- Visual Filter: filter using a mini-chart (bar/line/donut) in filter bar
- KPI Tag: key metric badge shown above visual filter
- @UI.Chart: defines chart type, dimensions, measures — required for ALP
- @UI.PresentationVariant: sets default sort, group, visualisation for table
- Compact filter: regular filter bar alongside visual filter
- Chart and table stay in sync — filtering one updates the other

ALP Page Anatomy

KPI Tag	Visual Filter	Compact Filter	Chart Area	Table Area
`@UI.KPI` `@UI.DataPoint`	`@UI.Chart` `(bar/line/donut)`	`@UI.SelectionFields` `(text filters)`	`@UI.Chart` `@UI.PresentationVariant`	`@UI.LineItem` `@UI.SelectionVariant`

⚠ Exam Trap ★ **ALP requires SAP HANA database. List Report works with Any DB. The exam will test this.**

Know These Cold — C_FIORD Exam

U1–U3 — FE Overview + Tools + Templates

- FE = annotation-driven — NO hand-coded XML views needed
- CAP = Cloud Application Programming Model (Node.js/Java on BTP)
- OData V4 ONLY — CAP with Fiori Elements never uses V2
- 5 templates: List Report, Object Page, OVP, ALP, Worklist
- Fiori Tools: App Generator, Page Map, Page Editor, Guided Dev
- ALP requires SAP HANA — List Report works with Any DB

U4–U8 — List Report Annotations

- @UI.SelectionFields → filter bar fields
- @UI.LineItem → list report table columns
- @UI.DataFieldForAction → custom action button
- Bound action = needs row selection | Unbound = no selection needed
- Criticality: 1=red 2=amber 3=green 0=neutral (numbers not text)
- @Common.SideEffects → auto-refresh triggered by field change

U9–U14 — OP + FPM + ALP + Nav

- @UI.HeaderInfo → Object Page title/description
- @UI.Facets → body structure (sections/tabs)
- @UI.FieldGroup → form field groups in a section
- @FieldControl → dynamic editability (0/1/3/7)
- FPM: Custom Section / Column / Action (prefer annotations first)
- macros:Table / Chart / FilterBar = Building Blocks in custom sections

Exam Traps: FE ≠ Freestyle | CAP ≠ ABAP Gateway | OData V4 only (not V2) | ALP = HANA only | Bound ≠ Unbound action

Before Session 5 — Action Items

Session 5 covers UX406: Fiori Elements — Advanced Annotations & Extensions

Next Session: Session 5 – Advanced Fiori Elements (UX406) | 2 Hours

1

Complete UX405 on SAP Learning

All 14 units (9h 19min) — learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service

2

Do all UX405 exercises in SAP BAS

Create a CAP project, add a CDS model, generate a Fiori Elements app using the Application Generator in BAS

3

Write @UI.LineItem + @UI.Facets from memory

These two annotations appear in EVERY FE exam question. Know the syntax cold — without looking it up

4

Practice bound vs unbound action scenarios

For each scenario: does it need a selected row? Yes → bound. No → unbound. Drill this until automatic.

5

Study the Fiori Elements Annotation Reference

sapui5.hana.ondemand.com/sdk/#/topic/7619517a924f4e859801e3a5d6a5d9f5 — bookmark the @UI.* section

Questions?

Session 4 • UX405 • SAP Fiori Elements with CAP OData V4

UX405 on SAP Learning —	learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service
Fiori Elements V4 Reference —	sapui5.hana.ondemand.com/sdk/#/topic/7619517a924f4e859801e3a5d6a5d9f5
CAP Documentation —	cap.cloud.sap/docs
SAP Fiori Tools Docs —	help.sap.com/docs/sap-fiori-tools
SAP BTP Trial (BAS) —	account.hanatrial.ondemand.com
SAP Community Fiori —	community.sap.com/topics/fiori
Fiori Apps Library —	fioriappslibrary.hana.ondemand.com

Next: Session 5 – Advanced Fiori Elements Annotations (UX406)